HBA Delivery

Le domaine logistique partagé : architecture, services, contrats

Document d'entrée pour un développeur qui arrive sur le projet.
État du dépôt au 1er octobre 2026.

HBA Tech et Trade — Bénin (pilote Cotonou), zone UEMOA.
.NET 9 · PostgreSQL · Kafka · Redis · gRPC · Flutter · Next.js

Sommaire

1. Ce que fait le système, et ce qu'il ne fait pas p. 2
2. Le schéma d'ensemble p. 3
3. Les règles qui ne se négocient pas p. 4
4. Comment un service est bâti p. 5
5. Les services, un par chapitre p. 7
6. Les trois applications et la console p. 17
7. L'infrastructure p. 18
8. Ce qui reste ouvert p. 19
Annexe — les vingt et une décisions d'architecture p. 20

Comment lire ce document

Chaque chapitre de service commence par une fiche : son rôle en une phrase, ses agrégats, son contrat gRPC, les événements qu'il publie et consomme, son stockage. Vient ensuite ce qu'il faut comprendre pour y toucher — les décisions qui expliquent sa forme, et les pièges connus.

Les chiffres de taille sont mesurés, pas estimés : ce sont les fichiers `.cs` de chaque couche, tests exclus. Ils disent la densité d'un service, pas sa qualité — Realtime en compte zéro dans trois couches, et c'est un choix justifié au chapitre qui le concerne.

> LA RÉFÉRENCE UNIQUE
>
> `docs/architecture/referentiel-acteurs.md` est la source de vérité du projet : acteurs, rôles, matrice de visibilité. Le modèle de domaine, les contrats, les tests et les écrans en découlent. Ce qui n'y figure pas est listé dans `points-a-trancher.md` — quarante points à ce jour — plutôt que supposé. **Ce document explique le code ; il ne remplace pas le référentiel.**

1. Ce que fait le système, et ce qu'il ne fait pas

HBA Delivery exécute la **livraison physique** et, depuis le 1er octobre 2026, le **transport de personnes**. Il ne gère pas la commande commerciale : c'est HBA Express, HBA Food ou un partenaire externe qui vend, et qui demande ensuite une course au système.

Trois contraintes de marché qui ont façonné le code

UNE ADRESSE EST UN POINT GPS, UN REPÈRE ÉCRIT ET UN TÉLÉPHONE

L'adressage formel n'est pas fiable à Cotonou. Cette contrainte ne s'arrête pas à l'interface : le type `Location` du domaine *refuse* une adresse sans repère. Aucun écran ne demande de numéro de rue, et les deux applications mobiles choisissent leurs points sur une carte.

LES MONTANTS SONT DES ENTIERS DE FRANCS CFA

Pas de décimales, pas de flottants, nulle part — ADR 0006. Les taux sont exprimés en **points de base**, et deux champs de la grille tarifaire en utilisent avec des sens opposés : la part du livreur est une fraction (8000 = 80 %), la majoration est un multiplicateur (10000 = ×1,00, c'est-à-dire aucune majoration).

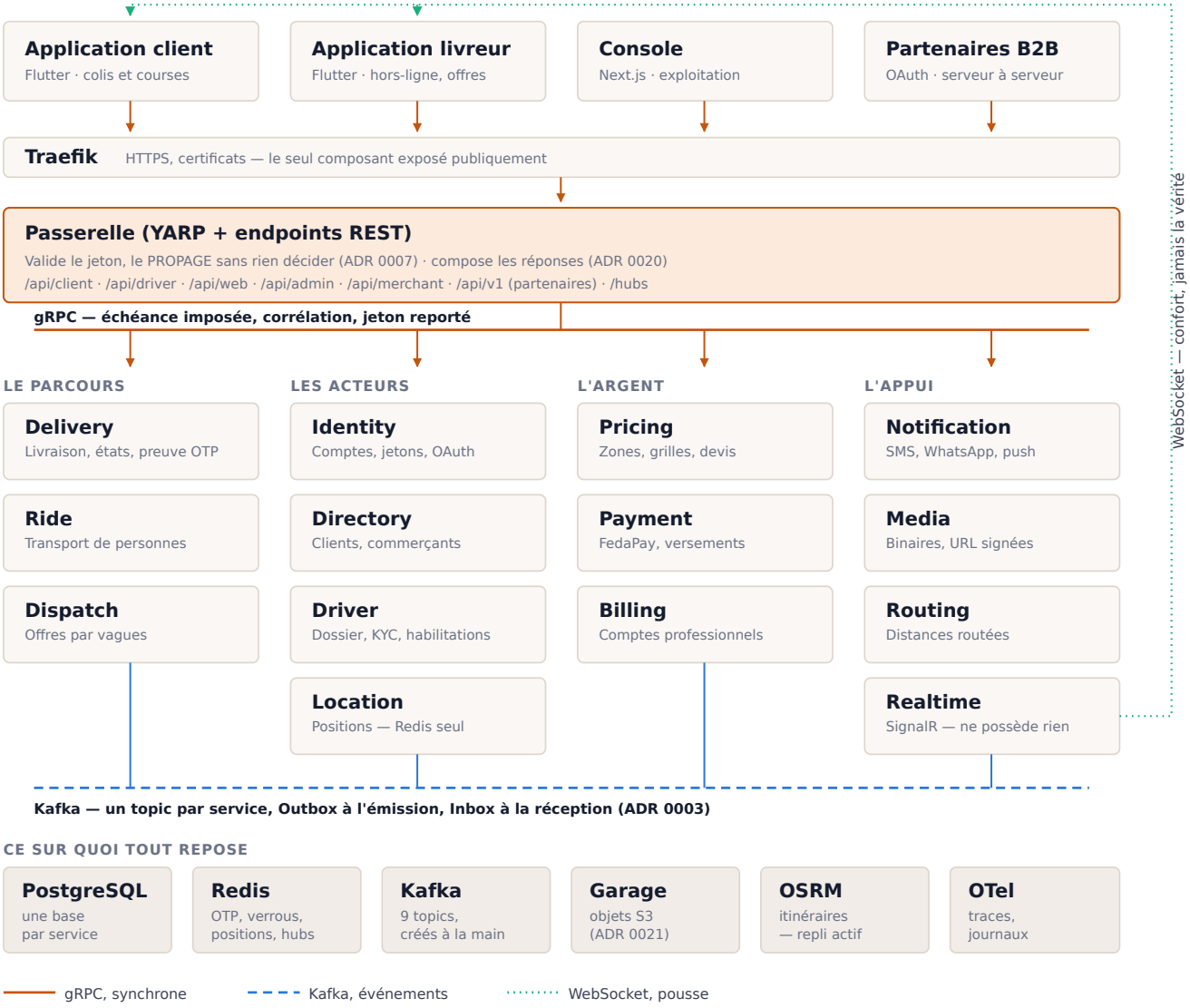
LE RÉSEAU COUPE, ET LE TÉLÉPHONE DOIT CONTINUER

L'application livreur met ses actions en file quand le réseau manque et les rejoue ensuite, avec leur horodatage d'origine. Le temps réel (SignalR) est un *confort* ; le sondage toutes les trente secondes reste la *vérité*. Le service de diffusion consomme Kafka au plus récent et ne rejoue rien : supprimer le sondage échangerait un écran lent contre un écran faux.

Le périmètre, en une table

LE SYSTÈME FAIT	LE SYSTÈME NE FAIT PAS
Créer et suivre une livraison de colis	Vendre un produit ou un repas
Créer et suivre une course de personnes	Tenir un catalogue
Chiffrer le prix et le figer	Gérer un stock
Chercher un livreur et lui offrir la mission	Noter un livreur *(service Rating, plus tard)*
Encaisser, rembourser, reverser	Traiter un litige *(service Support, plus tard)*
Tenir le dossier et le KYC des livreurs	Produire la comptabilité générale
Prévenir par SMS, WhatsApp et push	

2. Le schéma d'ensemble



Aucun service gRPC n'est exposé publiquement. Aucun service ne lit la base d'un autre, même en lecture : ce qu'il veut savoir, il le demande en gRPC ou l'apprend par un événement.

Quatorze services, une passerelle, trois interfaces. Aucun service gRPC n'est exposé publiquement : tout passe par Traefik puis par la passerelle, qui valide le jeton et le *propage* sans rien décider.

3. Les règles qui ne se négocient pas

1. L'autorisation se vérifie côté service

La passerelle valide le jeton mais ne décide rien : elle le propage, et le service le revalide puis applique lui-même la matrice de visibilité (ADR 0007). Le jeton qui atteint un service est celui de *l'utilisateur final*, reporté par un intercepteur gRPC — pas celui d'un service de confiance. Sans cette double vérification, tout porteur d'un jeton valide atteignant un service directement pourrait écrire une grille tarifaire.

L'exception est explicite : un appel descendant d'un événement Kafka n'a pas d'utilisateur derrière lui, et utilise un **jeton de service** émis par Identity (ADR 0018).

2. Le prix est figé à la confirmation

Un changement de grille ne touche jamais une course déjà confirmée. D'où le `PricingSnapshot` *recopié* dans la livraison, et non une référence au devis (ADR 0004). La même règle explique pourquoi une grille ne se modifie jamais : on en crée une version et on clôt la précédente à la seconde où la nouvelle prend effet — jamais de recouvrement, jamais de trou.

3. Seul le livreur livre

La transition vers `Delivered` n'existe que pour l'acteur `driver`, et seulement avec un OTP valide dicté par le destinataire (ADR 0005). Un administrateur peut clore en `Failed` ou `Cancelled`, jamais en `Delivered`. **Deux tests d'architecture échouent** si quelqu'un modifie la table des transitions dans ce sens.

> CE QUE CES RÈGLES COÛTENT, ET POURQUOI ON LES PAIE
>
> Vérifier deux fois l'autorisation double le code de contrôle. Recopier le prix duplique des données. Interdire à l'administrateur de clore en « livré » oblige le support à expliquer au client pourquoi il ne peut pas « juste valider ». Les trois sont des coûts assumés : ils rendent impossible une classe entière d'erreurs qui, sinon, ne se verraient qu'à la facture.

Deux présences, et il ne faut pas les confondre

« Je me déclare disponible » est un **état métier** : il se décide, il s'audite, il survit à un téléphone éteint — il vit dans **Driver**. « Mon téléphone émet » est un **fait technique** qui s'évapore tout seul au bout de la fenêtre de fraîcheur — il vit dans **Location**. Pour une vague de dispatch, « disponible » est la *conjonction des deux*, et c'est l'appelant qui croise : Location dit qui est proche, Driver dit qui est éligible.

4. Comment un service est bâti

Quatre projets, une dépendance qui ne remonte jamais

PROJET	CE QU'IL CONTIENT	CE QU'IL NE CONNAÎT PAS
DOMAIN	Agrégats, objets-valeur, événements de domaine, invariants	EF Core, gRPC, Kafka, HTTP
APPLICATION	Commandes, requêtes, handlers, vues, ports, autorisation	Toute implémentation technique
INFRASTRUCTURE	EF Core, dépôts, clients gRPC, consommateurs, adaptateurs	—
API	Service gRPC, hôte, santé, configuration	Les règles métier

Des **tests d'architecture** vérifient ces frontières. Le domaine n'a aucune dépendance externe : c'est pourquoi le polygone d'une zone tarifaire vit dans une colonne `geography` et non dans l'agrégat `Zone` — faire de la géométrie dans le domaine aurait demandé une bibliothèque que les tests interdisent.

Un CQRS fait maison, et volontairement petit

Pas de MediatR. Un `IDispatcher` avec deux méthodes : `SendAsync` pour un `ICommand<T>`, `QueryAsync` pour un `IQuery<T>`. Les handlers sont découverts par assembly au démarrage. Le gain n'est pas la performance : c'est qu'on peut lire le dispatcher en entier en une minute.

Les refus sont des exceptions, sauf dans Ride

`DomainException(code, message)` porte un code métier stable que les applications branchent — `NO_TARIFF`, `QUOTE_SERVICE_MISMATCH`, `OVERLAPPING_TIME_BAND`. `ForbiddenException` et `NotFoundException` complètent. Un intercepteur gRPC les traduit en statuts, et la passerelle en codes HTTP : `FailedPrecondition` devient 409, `InvalidArgument` devient 400, et tout ce qui n'est pas traduit devient 500 — ce qui est le signal qu'il manque une traduction.

Outbox à l'émission, Inbox à la réception

Un événement de domaine n'est pas publié directement : il est écrit dans la table `outbox` *dans la même transaction* que la modification qui l'a produit, puis repris par un dispatcher (ADR 0003). À la réception, une table `inbox` garantit qu'un message rejoué ne produit pas deux effets. **La clé de partition est toujours l'identifiant de l'agrégat**, ce qui garantit l'ordre des événements d'une même livraison.

Un topic par service, nommé `hba.<service>.events.v1`, et le type de l'événement voyage dans l'en-tête. Neuf topics existent ; ils sont créés explicitement au démarrage de l'infrastructure, car le broker refuse la création automatique.

Le temps, et pourquoi il a son propre service

`ITimeCalendar` (ADR 0019) porte le fuseau métier — `Africa/Porto-Novo`, UTC+1 toute l'année — et le découpage des fenêtres en demi-ouvert. Aucun service ne déduit l'heure locale lui-même :

une grille de nuit valable 22 h → 6 h s'écrit en minutes depuis minuit *heure locale*, et c'est le calendrier qui convertit.

5. Les services, un par chapitre

Quatorze services, chacun avec sa base, son déployable et son topic. Aucun ne lit la base d'un autre, même en lecture : ce qu'un service veut savoir, il le demande en gRPC ou l'apprend par un événement.

SERVICE	RÔLE EN UNE PHRASE	DOMAINE	APPLI.	INFRA.	MIGR.
IDENTITY	Comptes, jetons, partenaires	16	27	11	11
DIRECTORY	Profils clients, commerçants	10	16	5	10
DELIVERY	La livraison de colis	15	32	12	9
RIDE	Le transport de personnes	10	13	7	4
PRICING	Zones, grilles, devis	13	17	9	10
DISPATCH	Les offres par vagues	7	16	9	8
DRIVER	Le dossier du livreur	8	21	6	8
LOCATION	Où sont les livreurs	5	6	4	0
PAYMENT	Encaissements, versements	9	31	18	11
BILLING	Comptes des donneurs d'ordre	7	11	5	1
NOTIFICATION	SMS, WhatsApp, push	7	7	13	7
MEDIA	Les binaires	2	5	4	1
ROUTING	Distances et durées	5	4	5	0
REALTIME	Pousser au lieu de sonder	0	0	0	0

Nombre de fichiers `.cs` par couche, tests exclus. Zéro migration veut dire *aucune base relationnelle*, et c'est voulu pour Location, Routing et Realtime.

Identity — comptes, jetons, partenaires

AGRÉGATS	`Account`, `RefreshToken`, `PartnerClient`
CONTRAT	23 appels : OTP, mot de passe, jetons, rôles, suppression de compte, clients OAuth des partenaires, jeton de service
PUBLIE	`AccountRegistered`, `AccountRolesChanged`, `AccountStatusChanged`, `AccountLinkedToDriver`, `AccountDeletionRequested`, `AccountErased`, `PartnerClientRegistered` … (11 événements)
CONSOMME	rien
STOCKAGE	PostgreSQL `hba_identity` · Redis (codes OTP) · volume pour la clé de signature

Identity est le seul service qui **émet** des jetons, et le seul que les autres n'interrogent presque jamais : ils vérifient une signature contre le JWKS qu'il publie, sans aller lui demander quoi que ce soit. Les jetons sont RS256, le rafraîchissement est à usage unique (ADR 0010) — rejouer un jeton de rafraîchissement révoque toute la lignée, ce qui transforme un vol de jeton en déconnexion visible plutôt qu'en accès silencieux.

Les codes de connexion vivent dans Redis, pas en base (ADR 0011). Ils durent quelques minutes, se comptent par tentatives et disparaissent d'eux-mêmes : les écrire en base aurait créé une table dont 99,9 % des lignes sont mortes, et une donnée personnelle à purger. Le canal suit l'ADR 0014 : WhatsApp porte le code de connexion quand le client y a consenti, le SMS reste seul vers le destinataire d'un colis — qui n'a ni compte ni consentement.

Un compte n'est pas un profil. Identity publie `AccountRegistered`, Directory en crée le profil. Le destinataire d'un colis n'a ni l'un ni l'autre, et ne doit jamais s'en voir créer un : c'est la confusion la plus coûteuse à laisser s'installer.

> SANS SA CLÉ, IDENTITY DÉMARRE ET NE SERT À RIEN
>
> Hors développement, `JwtSigning:AllowEphemeralKey` vaut faux : une clé éphémère invaliderait tous les jetons à chaque redémarrage. Le service refuse donc d'en générer une — mais *tard*, à la première émission. Il répond à `/health`, sert le JWKS, accepte les demandes de code, et s'effondre à la première connexion. D'où la cible `make identity-key`.

Directory — profils clients, commerçants, points de collecte

AGRÉGATS	`Customer`, `Merchant`
CONTRAT	19 appels : profil et adresses favorites, photo, journal d'accès, commerçants et points de collecte
PUBLIE	`CustomerProfileCreated`, `CustomerProfileUpdated`, `CustomerPhotoChanged`, `MerchantCreated`, `MerchantUpdated`, `MerchantDeactivated`, `PickupPointChanged`
CONSOMME	Identity (`IdentityEventsConsumer`)
STOCKAGE	PostgreSQL `hba_directory`

Un seul service pour le client particulier *et* le commerçant, alors que le référentiel laissait le choix (ADR 0009). La raison : les deux sont des *annuaires* — un nom, un téléphone, des lieux favoris — et deux services auraient partagé le même modèle de lieu, la même règle de repère obligatoire et le même consommateur d'Identity.

Directory tient la **copie de travail** du nom et du téléphone ; la référence reste dans Identity. Ce n'est pas une duplication par négligence : un écran qui affiche vingt livraisons ne peut pas faire vingt appels à Identity, et un profil doit rester lisible quand Identity est indisponible.

Le journal d'accès est une fonctionnalité, pas un log. `GetCustomerAccessLog` dit qui, au back-office, a ouvert la fiche d'un client — avec la date. C'est la contrepartie du droit d'ops à voir des données personnelles.

Delivery — la livraison de colis

AGRÉGAT	`Delivery` — et il est le cœur du système
CONTRAT	11 appels : création, lecture, annulation, les quatre transitions du livreur, incident, clôture administrative, statistiques
PUBLIE	16 événements, de `DeliveryCreated` à `DeliveryCompleted`, en passant par `DriverSearchStarted`, `DriverAssigned`, `NoDriverFound`, `DeliveryOtpAttemptFailed`, `PreuveAttachee`
CONSOMME	Payment (le paiement confirme), Dispatch (l'offre acceptée affecte)
STOCKAGE	PostgreSQL `hba_delivery` (PostGIS) · 9 migrations

La machine à états d'une livraison est la pièce la plus relue du dépôt — elle a son propre document, `flux-livraison.md`. Les transitions ne sont pas des `if` dispersés : elles vivent dans une table, et chacune nomme l'acteur qui a le droit de la franchir. C'est cette table que deux tests d'architecture protègent.

Le prix est recopié, pas référencé (ADR 0004). La livraison porte un `PricingSnapshot` figé à la confirmation : le total, la part du livreur, la version de grille. Une grille qui change le lendemain ne touche pas une course d'hier, et un devis supprimé ne rend pas une facture illisible.

L'OTP du destinataire est la preuve de livraison (ADR 0005). Il est dicté au livreur, jamais affiché chez lui ni journalisé. Les tentatives ratées sont comptées et publiées — `DeliveryOtpAttemptFailed` —, parce qu'un livreur qui se trompe trois fois n'est pas au bon endroit.

Ride — le transport de personnes

AGRÉGAT	`RideAggregate`
CONTRAT	10 appels : demande, lecture, courses actives, historique, et les cinq transitions du chauffeur
PUBLIE	`RideRequested`, `RideDriverSearchStarted`, `RideDriverAssigned`, `RideDriverArriving`, `RideDriverArrived`, `RideStarted`, `RideCompleted`, `RideCancelled`, `RideNoDriverAvailable`, `RidePaymentFailed`
CONSOMME	Payment, Dispatch
STOCKAGE	PostgreSQL `hba_ride` · 4 migrations

Ride est le plus jeune service, né de la décision de mobilité du 1er octobre 2026. Il **n'est pas une copie de Delivery** : un passager n'a ni destinataire, ni code de remise, ni colis. Il monte, il descend. C'est pourquoi aucune preuve de fin de course n'existe — et en inventer une l'aurait inscrite dans l'interface avant que quiconque l'ait décidée.

Le prix est encaissé avant la recherche d'un chauffeur. Une course impayée n'est jamais dispatchée, et sort même du périmètre que le service rend au passager : elle existe, elle est lisible par son propriétaire, mais elle ne figure dans aucune liste tant qu'elle n'est pas payée.

La course porte un **instantané du chauffeur** — nom, téléphone, marque, modèle, couleur, plaque — posé une seule fois à l'affectation. Avant, chaque lecture interrogeait Driver : un écran de suivi qui rafraîchit toutes les cinq secondes faisait cinq appels par minute et par passager, et Driver indisponible une seconde effaçait le nom.

Pricing — zones, grilles, devis

AGRÉGATS	`Zone`, `Tariff`, `Quote`
CONTRAT	9 appels : devis (obtenir, relire, consommer), grilles (créer, clore, lister), zones (lister, enregistrer, activer)
PUBLIE	`hba.pricing.events.v1`
CONSOMME	rien
STOCKAGE	PostgreSQL `hba_pricing` **avec PostGIS** · 10 migrations

Pricing est la seule autorité sur le prix. Ni le client, ni le livreur, ni le commerçant ne le fixent : le service ne prend aucun montant en entrée — seulement deux points, une nature, un véhicule et un poids — et rend un devis qui sera consommé tel quel.

La clé d'une grille est un **quadruplet** : zone, nature (colis ou course), véhicule, plage horaire. Une grille ne se modifie jamais : on en crée une version et la précédente est close à la seconde où la nouvelle prend effet. Un contrôle applicatif refuse deux plages qui se recouvrent sur la même clé — sans lui, le prix d'une course à 23 h dépendrait de l'ordre de lecture en base.

Les zones sont de vrais polygones depuis le 1er octobre 2026. Le contour vit dans une colonne `geography(Polygon,4326)` et *seule la requête le manipule* : `ST_Covers` — et non `ST_Contains`, qui n'existe pas pour le type `geography` — départagé par priorité décroissante. Accepter le bord est voulu : deux zones mitoyennes ne doivent pas laisser une bande d'un mètre sans prix entre elles.

> LE RÉGLAGE À NE PAS OUBLIER EN PRODUCTION
>
> `Zones:FallbackZoneCode` fait entrer le monde entier dans une zone tant qu'aucun polygone ne couvre un point. C'est commode en développement et faux en production : il doit être **vide**, pour qu'un point hors zone soit refusé par `ZONE_NOT_COVERED` au lieu d'être facturé au tarif d'une ville où il n'est pas.

Dispatch — les offres par vagues

AGRÉGAT	`DispatchAggregate` — il porte le couple `(ServiceType, ReferenceId)`
CONTRAT	6 appels : accepter, refuser, offre courante, offre manuelle, réaffectation, statistiques
PUBLIE	`OfferSent`, `OfferAccepted`, `OfferDeclined`, `OfferExpired`, `OfferSuperseded`, `DispatchExhausted`
CONSOMME	Delivery, Ride
STOCKAGE	PostgreSQL `hba_dispatch` · Redis (verrou de réservation) · 8 migrations

Dispatch ne sait pas ce qu'est un colis ni un passager : il connaît une **mission**, désignée par le couple `(nature, référence)`. Ce couple a remplacé partout l'ancien `delivery_id` le 1er octobre 2026 — le type dit à qui appartient l'identifiant, et c'est lui qui évite qu'un consommateur de livraisons agisse sur une course.

Un livreur à la fois, le plus proche d'abord. Une vague propose la mission à un seul livreur, avec une échéance ; s'il refuse ou laisse expirer, la vague suivante part. L'alternative — proposer à dix livreurs et garder le premier qui accepte — produit neuf déçus par course et une course de vitesse entre applications.

Le croisement se fait ici : Dispatch demande à Location qui est proche (avec une sur-interrogation de cinq, parce que beaucoup seront filtrés), puis à Driver lesquels sont vérifiés, disponibles, habilités et motorisés comme il faut. Deux services, deux questions, une réponse.

À savoir : `ForceReassign` répond `UNIMPLEMENTED` volontairement. Réaffecter une course déjà acceptée touche à la politique d'annulation — le livreur en route est-il dédommagé, et sur quelle base ? C'est le point 3 des points à trancher, et il n'est pas tranché.

Driver — le dossier du livreur

AGRÉGAT	`DriverAggregate` : le dossier, le véhicule, les habilitations, l'état de travail
CONTRAT	17 appels : dossier et KYC, véhicule, habilitations, présence déclarée, filtrage pour le dispatch, profil public, statistiques
PUBLIE	`DriverRegistered`, `DriverVehicleDeclared`, `DriverCapabilitiesDeclared`, `DriverDocumentAttached`, `DriverApplicationSubmitted`, `DriverKycReviewed`, `DriverStatusChanged`…
CONSOMME	Identity, Delivery, Dispatch
STOCKAGE	PostgreSQL `hba_driver` · pièces dans Garage via Media · 8 migrations

La position du livreur n'est plus ici depuis le 1er octobre 2026. Elle vit dans Location. Driver garde la présence *déclarée* — « je me mets en ligne » — qui est un état métier audité ; la position technique s'évapore ailleurs. Driver ne dépend plus de Redis du tout.

Un dossier validé se fige. Laisser remplacer une CNI après validation reviendrait à valider une personne et à en laisser travailler une autre. Les pièces, le véhicule *et* les habilitations ne se modifient donc plus une fois le dossier validé : la correction passe par ops, qui suspend d'abord.

Les habilitations disent le métier, le véhicule dit la possibilité. Un chauffeur habilité au transport de personnes qui roule aujourd'hui en vélo ne peut pas prendre de passager : c'est le véhicule déclaré qui tranche, pas le dossier. Et le colis est le *socle* — il ne se déclare pas, un dossier vide n'est pas un livreur sans métier.

Location — où sont les livreurs

AGRÉGAT	aucun — et c'est le propos
CONTRAT	5 appels : mettre à jour, lire, chercher à proximité, lister les positions fraîches, oublier
PUBLIE	des battements sur un canal Redis (pas de topic Kafka)
CONSOMME	rien
STOCKAGE	**Redis seulement** : un index GEO, un ensemble trié pour la fraîcheur, un hachage pour le mouvement. Aucune base relationnelle, aucune migration.

Location répond à une seule question : *où se trouve actuellement ce chauffeur ?* Il ne sait ni son nom, ni son téléphone, ni son véhicule, ni son dossier. **Sa colonne dans la table « où vit quoi » est presque entièrement faite de « non », et c'est son intérêt** : il garde la trace de l'endroit où se tiennent des gens sans constituer un fichier de personnes.

Trois champs que le téléphone donnait depuis toujours et que personne ne transportait — **cap, vitesse, précision** — y arrivent enfin. Deux d'entre eux existaient même au contrat gRPC de Driver sans que personne ne les remplisse : c'est pour cela qu'aucune carte n'avait jamais pu orienter une flèche.

Le nul n'est pas le zéro. Un téléphone à l'arrêt ne rend aucun cap ; zéro degré veut dire « plein nord ». Les trois champs sont donc facultatifs de bout en bout, et un champ absent du corps JSON dit au serveur « le téléphone n'a rien dit » — c'est cette différence qui empêche toutes les motos immobiles de pointer vers le nord sur la carte de l'exploitation.

Payment — encaissements, remboursements, versements

AGRÉGATS	`PaymentIntent`, `DriverLedgerEntry`, `PayoutRequest`, `RefundDue`
CONTRAT	14 appels : intentions, remboursements, relevé et versements des livreurs, remboursements dus, statistiques
PUBLIE	`PaymentIntentSucceeded`, `PaymentIntentFailed`, `PaymentIntentRefunded`
CONSOMME	Delivery, Ride
STOCKAGE	PostgreSQL `hba_payment` · 11 migrations

Le fournisseur n'apparaît pas dans le domaine. FedaPay vit derrière un port de la couche Application, et l'agrégat ne connaît que des références opaques (ADR 0017). Le paiement se fait en *page hébergée* : l'application ouvre une adresse fournie par l'agrégateur, et aucun identifiant bancaire ne traverse jamais le système.

Seul le webhook signé fait foi. Le retour de l'utilisateur sur la page de retour ne prouve rien — ni encaissé, ni refusé : il prouve seulement qu'il est allé au bout de l'écran. Les deux applications l'écrivent noir sur blanc à l'utilisateur (« paiement en cours de confirmation »), parce que c'est la chose la plus forte qu'on ait le droit d'écrire.

Payment porte aussi le **versement aux livreurs** : un grand-livre (`DriverLedgerEntry`), des demandes de versement avec approbation, et les remboursements dus qu'un humain solde. C'est le service le plus dense en infrastructure du dépôt — dix-huit fichiers — parce qu'un agrégateur de paiement est une surface d'intégration, pas une bibliothèque.

Billing — les comptes des donneurs d'ordre

AGRÉGAT	`BillingAccount`
CONTRAT	5 appels : débiter, créditer, contre-passer, lire, ouvrir
PUBLIE	`BillingAccountOpened`, `AccountDebited`, `AccountCredited`, `CreditLimitGranted`, `LowBalanceReached`, `AccountStatusChanged`
CONSOMME	rien
STOCKAGE	PostgreSQL `hba_billing` · 1 migration

Billing tient les comptes des **professionnels** — commerçants et partenaires — et rien d'autre. Il ne connaît ni les courses, ni les livreurs, ni les clients particuliers. Il n'encaisse pas non plus : c'est Payment qui parle à l'agrégateur, et Billing qui enregistre ce que l'encaissement a produit.

Un seul modèle, un seul paramètre (ADR 0013). Le prépayé et le postpayé ne sont pas deux modules : ce sont deux valeurs de `SettlementMode` et une seule règle de débit, `solde + plafond ≥ montant`. En prépayé le plafond vaut zéro, donc le solde reste positif ; en postpayé il descend sous zéro, et ce solde négatif *est* l'encours du mois.

Pour le pilote, **seul le prépayé est en service**, et la recharge se fait à la main : finance constate un virement et crédite le compte. Ouvrir une recharge par mobile money aurait demandé de généraliser l'intention de paiement, qui refuse un paiement sans livraison — et une recharge n'a pas de course.

Notification — SMS, WhatsApp, push

AGRÉGAT	`SentNotification` — le journal des envois
CONTRAT	aucun service gRPC : **Notification écoute, il ne répond pas**
PUBLIE	rien
CONSOMME	un consommateur de commandes de notification
STOCKAGE	PostgreSQL `hba_notification` · 7 migrations

C'est le service le plus déséquilibré du dépôt, et volontairement : sept fichiers de domaine pour **treize** d'infrastructure. Un envoi de SMS n'a presque pas de règles ; il a des fournisseurs, des formats, des erreurs et des reprises. Les expéditeurs sont OVH pour le SMS et WhatsApp Cloud pour WhatsApp, derrière une même interface.

Un expéditeur non configuré ne fait pas tomber le service. Il y a un `UnconfiguredSender` et un rapport au démarrage qui dit ce qui est branché — parce qu'un service de notification silencieux est indiscernable d'un service qui marche, jusqu'au jour où un client n'a pas reçu son code.

Le canal dépend du destinataire, pas de la commodité (ADR 0014). WhatsApp porte le code de connexion d'un client qui y a consenti ; le SMS reste seul vers le destinataire d'un colis, qui n'a ni compte, ni consentement, ni application.

Media — les binaires

AGRÉGAT	`MediaAsset`
CONTRAT	5 appels : lire, obtenir une URL de lecture, lister, supprimer, supprimer tout d'un propriétaire
STOCKAGE	PostgreSQL `hba_media` (les métadonnées) · **Garage**, compatible S3 (les octets) · 1 migration

Un seul service pour tous les binaires du système : pièces du dossier livreur, photos de profil, preuves de livraison (ADR 0021). Avant lui, chaque service inventait sa convention de clés et son client S3.

La clé de stockage ne sort jamais : elle serait devinable d'un livreur à l'autre. Ce qui sort est une **URL signée et courte**, avec son échéance — pour que l'écran sache recharger au lieu d'afficher une image cassée.

Garage plutôt que MinIO (ADR 0014 du stockage objet) : l'image ne contient que le binaire, pas de shell, et l'initialisation se pilote depuis l'hôte. C'est aussi pourquoi `make garage-init` existe.

Routing — distances et durées

AGRÉGAT	aucun : c'est un calculateur, pas un domaine
CONTRAT	2 appels : `CalculateRoute`, `CalculateMatrix`
STOCKAGE	aucun — pas de base, pas de migration

Routing existe pour une raison précise : **tarifer une course exige une distance routée**, pas une distance à vol d'oiseau. Toutes les courses qui contournent la lagune de Cotonou seraient sous-facturées autrement.

Deux fournisseurs derrière la même interface : **OSRM**, et **haversine** en repli. Le second est un choix de développement assumé — préparer la carte du Bénin pour OSRM demande un prétraitement long — mais il faut le savoir : tant qu'il est actif, *tout prix du système repose sur une distance à vol d'oiseau*. C'est le point 36 des points à trancher : un prix ferme calculé sur une mesure dégradée.

À connaître aussi : le **profil d'OSRM est figé au prétraitement de la carte**, pas à l'appel. Changer de profil — voiture, moto, vélo — demande de refaire la carte, pas de changer un paramètre.

Realtime — pousser au lieu de sonder

AGRÉGAT	aucun. **Un seul projet, là où les autres en ont quatre.**
CONTRAT	pas de gRPC : trois hubs SignalR — `/hubs/client`, `/hubs/driver`, `/hubs/ops`
CONSOMME	Delivery, Ride, Dispatch (Kafka, **sans inbox**) et un canal Redis pour les positions
STOCKAGE	aucun. Redis sert de plan arrière SignalR et d'annuaire mission → chauffeur

Un dossier Domain existe pour tenir des invariants dans le temps. Ce service n'en a aucun, parce qu'il ne **possède** rien : il lit deux bus et écrit dans des sockets. Si on l'éteint une heure, il ne manque aucune donnée au système, seulement du confort aux écrans.

Pas d'inbox non plus, et c'est le même raisonnement : une trame écrite deux fois dans une socket est inoffensive. Les consommateurs partent donc au *plus récent* et commettent toujours — ce qui veut dire, en retour, qu'un changement d'état émis pendant une coupure est perdu pour toujours. D'où la règle énoncée au chapitre 1 : le direct est un confort, le sondage est la vérité.

La console d'exploitation reste au sondage, et ce n'est pas un oubli. SignalR depuis un navigateur exige le jeton en JavaScript — un WebSocket ne porte pas d'en-tête `Authorization` — alors que le jeton du back-office ne quitte jamais le serveur. Le donner au navigateur mettrait un jeton d'administration à portée de n'importe quelle injection, pour gagner quinze secondes sur une carte qui ne peut pas être plus fraîche que les points qu'elle affiche.

La passerelle

RÔLE	Le seul point d'entrée. Traduit REST/JSON en gRPC, valide le jeton, le propage, compose les réponses
PUBLICS	`/api/client`, `/api/driver`, `/api/web` et `/api/admin`, `/api/merchant`, `/api/v1` (partenaires), `/hubs` (relais SignalR via YARP)
STOCKAGE	aucun

Un seul déployable à la place de quatre BFF (ADR 0016). Les quatre publics partageaient le même pipeline d'authentification, les mêmes clients gRPC et la même traduction d'erreurs : quatre conteneurs auraient quadruplé l'exploitation pour séparer des fichiers que des dossiers séparent déjà.

Elle compose, elle ne décide pas (ADR 0020). Chaque service agrège ce qu'il possède ; la passerelle rapproche. Un exemple : la carte des livreurs de la console demande les positions fraîches à Location, puis les identités correspondantes à Driver, et assemble.

> UN PIÈGE À CONNAÎTRE AVANT D'AJOUTER UNE ROUTE
>
> La plupart des routes relaient le message protobuf tel quel, et `System.Text.Json` sérialise alors les énumérations en **entiers**. Les routes plus récentes mettent en forme à la main et rendent des *noms*. Les deux conventions coexistent ; les clients acceptent les deux orthographes là où ça compte. Ce piège a déjà coûté un écran de livreur muet.

6. Les trois applications et la console

Application cliente — Flutter

Envoi de colis et, depuis le 1er octobre 2026, commande de course. Deux points sur une carte, un véhicule, un prix ferme, un paiement en page hébergée, puis un suivi qui mêle sondage et temps réel. L'écran de suivi affiche la position du livreur affecté — décision du point 37 —, avec l'**âge de la position** écrit dessous : une épingle immobile se lit de deux façons, et l'écran doit dire laquelle.

Application livreur — Flutter

Dossier et KYC, mise en ligne, offres, missions. Elle est la plus exigeante des trois : elle fonctionne hors réseau en mettant ses actions en file, elle envoie une position enrichie, et elle sépare deux gestes qui n'ont pas le même poids — **accepter se glisse, refuser s'appuie**. Accepter engage le livreur devant un client ; un téléphone secoué dans une poche ne doit pas pouvoir le faire.

Console d'exploitation — Next.js 16, React 19

Commandes, livreurs, clients, versements, facturation, tarifs, zones, paramètres. Le jeton ne quitte jamais le serveur : une procuration Next.js le pose elle-même et ne relaie que trois préfixes — « une procuration qui colle un jeton d'administrateur sur n'importe quel chemin est un député confus ».

Un noyau partagé, et un système visuel

Les deux applications Flutter partagent `hba_core` (client HTTP, coffre à jetons, temps réel) et `hba_ui` (le style : relief et creux, couleurs, espacements). La règle visuelle tient en une phrase : *ce qui dépasse s'appuie, ce qui est creux se remplit*. L'action principale garde un aplat de couleur, parce que si tout est en relief, plus rien ne se distingue.

7. L'infrastructure

BRIQUE	RÔLE	À SAVOIR
TRAEFIK	Entrée HTTPS, certificats	Le seul composant exposé publiquement
POSTGRESQL	Une base par service	Image `postgis` : la géométrie est disponible partout, utilisée par Pricing et Delivery
REDIS	OTP, verrous, positions, plan arrière SignalR	Quatre usages sans rapport entre eux, un seul exemplaire
KAFKA	Les neuf topics d'événements	Création automatique **désactivée** : un topic absent est un silence, pas une erreur
GARAGE	Stockage objet compatible S3	Pas de shell dans l'image ; initialisation depuis l'hôte
OSRM	Itinéraires	Carte du Bénin non préparée à ce jour — repli haversine
OTEL	Traces et journaux	Corrélation propagée de bout en bout

Chaque service est un projet compose autonome

Quatorze services, quatorze fichiers `docker-compose.yml`, tous rattachés à un réseau externe `hba-internal` créé hors compose. Aucun service ne publie de port : seule la passerelle est jointe, par Traefik. Deux ports internes par service : **8080 en HTTP/1 pour REST, 8081 en HTTP/2 pour gRPC**.

Les migrations ne s'appliquent pas au démarrage hors développement (`Database:AutoMigrate` vaut faux) : un service ne touche pas au schéma de sa base en démarrant. C'est `make migrate` qui les applique — et l'oublier est un état normal d'un environnement neuf, pas un incident.

8. Ce qui reste ouvert

Rien de ce qui suit n'est caché dans le code : tout est écrit dans `points-a-trancher.md`, qui compte quarante points — chacun avec ses options et, pour les tranchés, la raison du choix et ce qui a été écarté.

DES DÉCISIONS EN ATTENTE

- **La politique d'annulation** (point 3) : qui dédommage le livreur déjà en route, et sur quelle base. Tant qu'elle n'est pas prise, `ForceReassign` répond `UNIMPLEMENTED`.
- **Un prix ferme sur une mesure dégradée** (point 36) : voir Routing.
- **La liste des motifs d'incident** (point 34) : la figer dans un écran la figerait dans le contrat et dans la base, avant que personne l'ait décidée.
- **Le mot « course »** (point 39) : il désigne le transport de personnes dans le référentiel, et une commande de colis dans l'application cliente.

DES CHANTIERS IDENTIFIÉS

- **Aucun test dans la console**, et aucun dans Dispatch, Payment, Location, Realtime, Ride, Routing.
- **Le commerçant n'a pas d'écran** : le service le gère, la passerelle l'expose, et personne ne peut en créer un depuis la console.
- **L'identité des paquets mobiles** : l'application cliente porte encore `com.example`, ce qui bloque sa publication.
- **Pas de reprise de paiement** : une commande dont le paiement a été interrompu n'a pas de route pour reprendre.

> LA RÈGLE QUI TIENT TOUT CELA ENSEMBLE
>
> Ce qui manque est *écrit* plutôt que supposé. Un point à trancher non implémenté vaut mieux qu'une règle inventée qui aura l'air juste pendant six mois — et qu'on ne pourra plus distinguer d'une décision quand il faudra la changer.

Annexe — les vingt et une décisions d'architecture

Chaque ADR tient en une page dans `docs/adr/` : le contexte, la décision, et ce qu'elle coûte.

0001 Microservices plutôt que monolithe modulaire
0002 gRPC pour le synchrone, Kafka pour l'asynchrone
0003 Outbox à l'émission, Inbox à la réception
0004 Le prix est recopié dans la livraison, pas référencé
0005 L'OTP du destinataire est la preuve de livraison
0006 Montants en entiers de francs CFA
0007 L'autorisation se vérifie côté service, jamais au BFF seul
0008 Un seul déployable par service
0009 Un seul service Directory, pas deux
0010 Jetons RS256, rafraîchissement à usage unique
0011 Les codes SMS vivent dans Redis, pas en base
0012 Une route publique ne dit rien de plus que nécessaire
0013 Un compte de facturation par donneur d'ordre
0014 WhatsApp porte le code, le SMS reste seul vers le destinataire
0015 HTTP+JSON pour les appels internes, à la place de gRPC
0016 Un seul gateway à la place des quatre BFF
0017 FedaPay comme agrégateur, en page hébergée
0018 Un service s'authentifie avec un jeton émis par Identity
0019 Fenêtre demi-ouverte, découpage dans le fuseau métier
0020 Chaque service agrège ce qu'il possède, la passerelle compose
0021 Garage pour les binaires, la passerelle porte les octets

Les documents à lire ensuite

REFERENTIEL-ACTEURS.MD	La référence unique : acteurs, rôles, matrice de visibilité
POINTS-A-TRANCHER.MD	Les quarante points, tranchés et non tranchés
FLUX-LIVRAISON.MD	La machine à états d'une livraison
MATRICE-VISIBILITE.MD	Qui voit quoi, et où c'est appliqué dans le code
DEMARRAGE-LOCAL.MD	Monter la pile sur un poste